LAPORAN PRAKTIKUM – BINARY TREE

Laporan ini disusun untuk memenuhi Tugas Ke Sembilan Praktikum Algoritma & Pemrograman

Dosen Pengampu : Dr. Ionia Veritawati, S.Si., MT.

Oleh :

Nama : Khairul Adam Efendi

NPM : 4525210035

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS PANCASILA

2025/2026

Soal Binary Tree_01 (nama file: PASD9-01.cpp):

- **Source Code**

1. Berdasarkan contoh no1 binary tree, dengan isi elemen data root dan daun/leaf melalui inputan (nama file : PASD9-01.ccp):

```
#include <iostream>
using namespace std;

typedef struct node *alamatnode;

typedef struct node {
   char INFO;
   alamatnode RIGHT;
   alamatnode LEFT;
} NODE;

typedef struct {
   NODE *root;
} TREE;

void BuatTree(char C, TREE *T) {
   NODE *Baru = new NODE;
   Baru->INFO = C;
   Baru->RIGHT = NULL;
   Baru->LEFT = NULL;
   T->root = Baru;
}

void TambahKanan(char C, NODE *root) {
   if (root->RIGHT == NULL) {
      NODE *Baru = new NODE;
      Baru->INFO = C;
      Baru->RIGHT = NULL;
```

```
        Baru->LEFT = NULL;
        root->RIGHT = Baru;
    } else {
        cout << "Sub Tree Kanan telah diisi" << endl;
    }
}

void TambahKiri(char C, NODE *root) {
    if (root->LEFT == NULL) {
        NODE *Baru = new NODE;
        Baru->INFO = C;
        Baru->RIGHT = NULL;
        Baru->LEFT = NULL;
        root->LEFT = Baru;
    } else {
        cout << "Sub Tree Kiri telah diisi" << endl;
    }
}

void CopyTree(NODE *root1, NODE **root2) {
    if (root1 != NULL) {
        *root2 = new NODE;
        (*root2)->INFO = root1->INFO;
        (*root2)->LEFT = NULL;
        (*root2)->RIGHT = NULL;

        CopyTree(root1->LEFT, &(*root2)->LEFT);
        CopyTree(root1->RIGHT, &(*root2)->RIGHT);
    } else {
        *root2 = NULL;
    }
}
```

```
bool isEqual(NODE *root1, NODE *root2) {
   if (root1 == NULL && root2 == NULL)
      return true;

   if (root1 == NULL || root2 == NULL)
      return false;

   return (root1->INFO == root2->INFO) &&
         isEqual(root1->LEFT, root2->LEFT) &&
         isEqual(root1->RIGHT, root2->RIGHT);
}

void CetakTreePreOrder(NODE *root) {
   if (root != NULL) {
      cout << root->INFO << " ";
      CetakTreePreOrder(root->LEFT);
      CetakTreePreOrder(root->RIGHT);
   }
}

void CetakTreeInOrder(NODE *root) {
   if (root != NULL) {
      CetakTreeInOrder(root->LEFT);
      cout << root->INFO << " ";
      CetakTreeInOrder(root->RIGHT);
   }
}

void CetakTreePostOrder(NODE *root) {
   if (root != NULL) {
      CetakTreePostOrder(root->LEFT);
      CetakTreePostOrder(root->RIGHT);
      cout << root->INFO << " ";
```

```
    }
}

int main() {
    TREE T;
    char root,V,W,Y,Z;
    cout<<"Masukkan data Root          :";cin>>root;
    cout<<"Masukkan data Leaf Kiri-Kiri : ";cin>>V;
    cout<<"Masukkan data Leaf Kiri-Kanan: ";cin>>W;
    cout<<"Masukkan data Leaf Kanan-Kiri: ";cin>>Y;
    cout << "Masukkan data Leaf Kanan-Kanan: ";cin>>Z;
    BuatTree(root, &T);
    TambahKiri('S', T.root);
    TambahKanan('U', T.root);
    TambahKiri(V, T.root->LEFT);
    TambahKanan(W, T.root->LEFT);
    TambahKiri(Y, T.root->RIGHT);
    TambahKanan(Z, T.root->RIGHT);
    cout << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~     PREORDER      ~~~~~~ " << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << endl;
    CetakTreePreOrder(T.root);
    cout << endl << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~     INORDER       ~~~~~~ " << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << endl;
    CetakTreeInOrder(T.root);
    cout << endl << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~     POSTORDER     ~~~~~~ " << endl;
```

```
        cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
        cout << endl;
        CetakTreePostOrder(T.root);
        cout << endl;
        return 0;
    }
```

- **SS Program**

```
#include <iostream>
using namespace std;

typedef struct node *alamatnode;

typedef struct node {
    char INFO;
    alamatnode RIGHT;
    alamatnode LEFT;
} NODE;

typedef struct {
    NODE *root;
} TREE;

void BuatTree(char C, TREE *T) {
    NODE *Baru = new NODE;
    Baru->INFO = C;
    Baru->RIGHT = NULL;
    Baru->LEFT = NULL;
    T->root = Baru;
}

void TambahKanan(char C, NODE *root) {
    if (root->RIGHT == NULL) {
        NODE *Baru = new NODE;
        Baru->INFO = C;
        Baru->RIGHT = NULL;
        Baru->LEFT = NULL;
        root->RIGHT = Baru;
    } else {
        cout << "Sub Tree Kanan telah diisi" << endl;
    }
}

void TambahKiri(char C, NODE *root) {
    if (root->LEFT == NULL) {
        NODE *Baru = new NODE;
```

```
        NODE *Baru = new NODE;
        Baru->INFO = C;
        Baru->RIGHT = NULL;
        Baru->LEFT = NULL;
        root->LEFT = Baru;
    } else {
        cout << "Sub Tree Kiri telah diisi" << endl;
    }
}

void CopyTree(NODE *root1, NODE **root2) {
    if (root1 != NULL) {
        *root2 = new NODE;
        (*root2)->INFO = root1->INFO;
        (*root2)->LEFT = NULL;
        (*root2)->RIGHT = NULL;

        CopyTree(root1->LEFT, &(*root2)->LEFT);
        CopyTree(root1->RIGHT, &(*root2)->RIGHT);
    } else {
        *root2 = NULL;
    }
}

bool isEqual(NODE *root1, NODE *root2) {
    if (root1 == NULL && root2 == NULL)
        return true;

    if (root1 == NULL || root2 == NULL)
        return false;

    return (root1->INFO == root2->INFO) &&
           isEqual(root1->LEFT, root2->LEFT) &&
           isEqual(root1->RIGHT, root2->RIGHT);
}

void CetakTreePreOrder(NODE *root) {
    if (root != NULL) {
```

```
        cout << root->INFO << " ";
        CetakTreePreOrder(root->LEFT);
        CetakTreePreOrder(root->RIGHT);
    }
}

void CetakTreeInOrder(NODE *root) {
    if (root != NULL) {
        CetakTreeInOrder(root->LEFT);
        cout << root->INFO << " ";
        CetakTreeInOrder(root->RIGHT);
    }
}

void CetakTreePostOrder(NODE *root) {
    if (root != NULL) {
        CetakTreePostOrder(root->LEFT);
        CetakTreePostOrder(root->RIGHT);
        cout << root->INFO << " ";
    }
}

int main() {
    TREE T;
    char root,V,W,Y,Z;
    cout<<"Masukkan data Root           :";cin>>root;
    cout<<"Masukkan data Leaf Kiri-Kiri : ";cin>>V;
    cout<<"Masukkan data Leaf Kiri-Kanan: ";cin>>W;
    cout<<"Masukkan data Leaf Kanan-Kiri: ";cin>>Y;
    cout << "Masukkan data Leaf Kanan-Kanan: ";cin>>Z;
    BuatTree(root, &T);
    TambahKiri('S', T.root);
    TambahKanan('U', T.root);
    TambahKiri(V, T.root->LEFT);
    TambahKanan(W, T.root->LEFT);
    TambahKiri(Y, T.root->RIGHT);
    TambahKanan(Z, T.root->RIGHT);
    cout << endl;
```

```
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~      PREORDER        ~~~~~~ " << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << endl;
    CetakTreePreOrder(T.root);
    cout << endl << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~      INORDER         ~~~~~~ " << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << endl;
    CetakTreeInOrder(T.root);
    cout << endl << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << " ~~~~~~      POSTORDER       ~~~~~~ " << endl;
    cout << " ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ " << endl;
    cout << endl;
    CetakTreePostOrder(T.root);
    cout << endl;
    return 0;
}
```

- **SS Running Program**

```
D:\ASD SMT2\ASD-A-25-26\PRAK9_Binary Tree_Asprak01_4525210035_Khairul Adam Efendi>PASD9-01.exe
Masukkan data Root          :Q
Masukkan data Leaf Kiri-Kiri : W
Masukkan data Leaf Kiri-Kanan: E
Masukkan data Leaf Kanan-Kiri: R
Masukkan data Leaf Kanan-Kanan: T

 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 ~~~~~~      PREORDER       ~~~~~~
 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

Q S W E U R T

 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 ~~~~~~      INORDER        ~~~~~~
 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

W S E Q R U T

 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 ~~~~~~      POSTORDER      ~~~~~~
 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

W E S R T U Q
```

Soal Binary Tree_02 (nama file: PASD9-02.cpp):

- **Source Code**

2. Jika terdapat TREE dengan urutan A, F, D, B, E dan B. Buat tampilan berdasarkan Binary Tree (nama file : PASD9-02.ccp):
    a. Bagaimana cara memasukkan M setelah Menghapus E
    b. Bagaimana cara memasukkan N setelah Menghapus B
    c. Bagaimana cara memasukkan O setelah menghapus D
    d. Bagaimana cara memasukkan P setelah menghapus F

```
#include<iostream>
using namespace std;

struct Node{
   char data;
   Node *left;
   Node *right;
};

Node *root=NULL;

Node* buatNode(char data){
   Node *baru=new Node;
   baru->data=data;
   baru->left=NULL;
   baru->right=NULL;
   return baru;
}

Node* insert(Node *root,char data){
   if(root==NULL)
      return buatNode(data);
   if(data<root->data)
      root->left=insert(root->left,data);
   else if(data>root->data)
      root->right=insert(root->right,data);
   return root;
```

```
}

Node* minimum(Node *root){
   while(root->left!=NULL)
      root=root->left;
   return root;
}

Node* hapus(Node *root,char data){
   if(root==NULL)
      return NULL;
   if(data<root->data)
      root->left=hapus(root->left,data);
   else if(data>root->data)
      root->right=hapus(root->right,data);
   else{
      if(root->left==NULL){
         Node *temp=root->right;
         delete root;
         return temp;
      }
      else if(root->right==NULL){
         Node *temp=root->left;
         delete root;
         return temp;
      }
      Node *temp=minimum(root->right);
      root->data=temp->data;
      root->right=hapus(root->right,temp->data);
   }
   return root;
}
```

```
void inorder(Node *root){
   if(root!=NULL){
      inorder(root->left);
      cout<<root->data<<" ";
      inorder(root->right);
   }
}

int main(){
   char data[]={'A','F','D','B','E','C'};
   for(int i=0;i<6;i++)
      root=insert(root,data[i]);
   cout<<"Inorder Awal : ";
   inorder(root);
   cout<<endl;
   cout<<endl;
   root=hapus(root,'E');
   root=insert(root,'M');
   cout<<"a. Hapus E, Insert M : ";
   inorder(root);
   cout<<endl;
   root=hapus(root,'B');
   root=insert(root,'N');
   cout<<"b. Hapus B, Insert N : ";
   inorder(root);
   cout<<endl;
   root=hapus(root,'D');
   root=insert(root,'O');
   cout<<"c. Hapus D, Insert O : ";
   inorder(root);
   cout<<endl;
   root=hapus(root,'F');
   root=insert(root,'P');
```

```
    cout<<"d. Hapus F, Insert P : ";
    inorder(root);
    cout<<endl;
    return 0;
}
```

- **SS Program**

```
#include<iostream>
using namespace std;

struct Node{
    char data;
    Node *left;
    Node *right;
};

Node *root=NULL;

Node* buatNode(char data){
    Node *baru=new Node;
    baru->data=data;
    baru->left=NULL;
    baru->right=NULL;
    return baru;
}

Node* insert(Node *root,char data){
    if(root==NULL)
        return buatNode(data);
    if(data<root->data)
        root->left=insert(root->left,data);
    else if(data>root->data)
        root->right=insert(root->right,data);
    return root;
}

Node* minimum(Node *root){
    while(root->left!=NULL)
        root=root->left;
    return root;
}

Node* hapus(Node *root,char data){
    if(root==NULL)
        return NULL;
```

```
    if(data<root->data)
        root->left=hapus(root->left,data);
    else if(data>root->data)
        root->right=hapus(root->right,data);
    else{
        if(root->left==NULL){
            Node *temp=root->right;
            delete root;
            return temp;
        }
        else if(root->right==NULL){
            Node *temp=root->left;
            delete root;
            return temp;
        }
        Node *temp=minimum(root->right);
        root->data=temp->data;
        root->right=hapus(root->right,temp->data);
    }
    return root;
}

void inorder(Node *root){
    if(root!=NULL){
        inorder(root->left);
        cout<<root->data<<" ";
        inorder(root->right);
    }
}

int main(){
    char data[]={'A','F','D','B','E','C'};
    for(int i=0;i<6;i++)
        root=insert(root,data[i]);
    cout<<"Inorder Awal : ";
    inorder(root);
    cout<<endl;
    cout<<endl;
     root=hapus(root,'E');
     root=insert(root,'M');
     cout<<"a. Hapus E, Insert M : ";
     inorder(root);
     cout<<endl;
     root=hapus(root,'B');
     root=insert(root,'N');
     cout<<"b. Hapus B, Insert N : ";
     inorder(root);
     cout<<endl;
     root=hapus(root,'D');
     root=insert(root,'O');
     cout<<"c. Hapus D, Insert O : ";
     inorder(root);
     cout<<endl;
     root=hapus(root,'F');
     root=insert(root,'P');
     cout<<"d. Hapus F, Insert P : ";
     inorder(root);
     cout<<endl;
     return 0;
}
```

- **SS Running Program**

```
D:\ASD SMT2\ASD-A-25-26\PRAK9_Binary Tree_Asprak01_4525210035_Khairul Adam Efendi>PASD9-02
Inorder Awal : A B C D E F

a. Hapus E, Insert M : A B C D F M
b. Hapus B, Insert N : A C D F M N
c. Hapus D, Insert O : A C F M N O
d. Hapus F, Insert P : A C M N O P
```